

Evolution of Sequence-to-Sequence Models

1. Sequence-to-Sequence Tasks

Some problems require mapping an **input sequence** to an **output sequence** that may be of different length and structure — for example:

- Machine translation** (“I am happy” → “Je suis heureux”)

- Summarization**

- Speech-to-text**

These are called **sequence-to-sequence (Seq2Seq)** tasks. They require models that can handle variable lengths, preserve order, and understand dependencies between elements across the sequence.

2. Early Sequence Models: RNNs

Recurrent Neural Networks (RNNs) were the first widely used models for sequence data. They process tokens step-by-step, passing a **hidden state** forward that summarizes what has been seen so far.

- Strength:** Naturally handles sequences and token order.

- Weakness:**

- **Vanishing gradients:** The influence of earlier tokens fades as the sequence grows.
- **Short memory:** Struggles to capture long-range dependencies.

3. LSTMs and GRUs: Fixing RNN Weaknesses

To address these issues:

- LSTMs (Long Short-Term Memory)** introduced:

- A **cell state** to carry long-term information.
- **Gates** (forget, input, output) to control what is remembered, updated, or output.
- Better at preserving information over longer sequences.

- GRUs (Gated Recurrent Units):**

- A simpler variant with only two gates (update, reset).
- Merged hidden and cell states into one.
- Fewer parameters, faster training, similar performance to LSTMs in many cases.

These gated RNNs made sequence modeling more effective, but **they still processed tokens sequentially** and struggled with tasks where input and output lengths differed significantly.

4. The Need for Encoder–Decoder Architectures

While LSTMs/GRUs improved memory, they still worked best when the input and output sequences were aligned in length and timing (e.g., labeling tasks).

For translation, summarization, and other Seq2Seq problems:

- The input must be **fully understood** before output starts.

- The output may be longer, shorter, or rearranged compared to the input.

The solution: split the process into two parts:

- Encoder:** Reads the entire input sequence, step-by-step, and produces a **context vector** summarizing it.

- Decoder:** Starts from the context vector and generates the output sequence, step-by-step.

Benefits:

- 1.Handles **different-length input/output** naturally.
- 2.Gives the encoder time to fully process the input before output generation.
- 3.Creates a clean separation between understanding (encoder) and producing (decoder).

5. The Context Vector Bottleneck

In the original encoder–decoder design:

- The **final hidden state** of the encoder (the context vector) was the *only* information passed to the decoder.

- This worked for short sequences, but in long or complex inputs, compressing all information into a single fixed-length vector caused an **information bottleneck** — some details were inevitably lost.

6. Attention Mechanism: Easing the Bottleneck

Attention was introduced to address the limitations of the single context vector.

- Instead of using only the final encoder state, attention lets the decoder **look back at all encoder hidden states** when generating each output token.

- At each step, the decoder calculates **attention weights** over the encoder states, focusing on the most relevant parts of the input.

- Impact:**

- Greatly improves performance on long sequences.
- Particularly effective for translation, where different output tokens align with different parts of the input.

Initially, attention was an **add-on** to RNN/LSTM/GRU encoder–decoders.

7. Transformers: Attention as the Foundation

The **Transformer** architecture took the attention idea further:

- Removed recurrence entirely.

- Built both encoder and decoder from **stacked self-attention layers** + feed-forward networks.

- Added **positional encodings** to preserve order (since self-attention alone doesn’t know token positions).

- Benefits:**

- Processes all tokens in parallel → much faster training.
- Handles long-range dependencies without vanishing gradients.
- Scales well to large datasets and deep networks.

At a high level, Transformers still follow the **encoder–decoder** pattern, but internally they rely entirely on **self-attention and cross-attention** instead of recurrent processing.

8. The Evolution Path

1.RNNs → basic sequence modeling, but poor long-term memory.

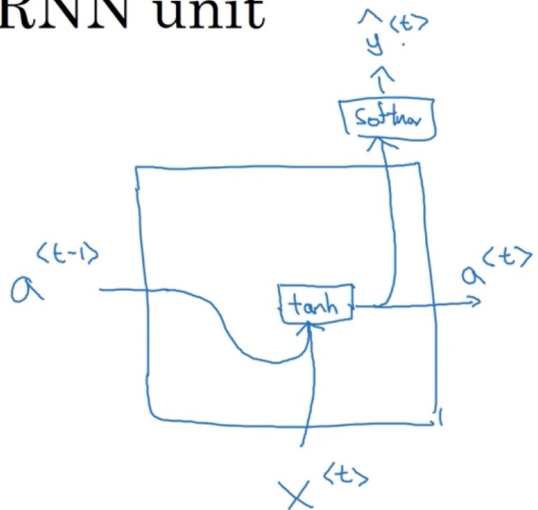
2.LSTMs / GRUs → improved memory with gates, but still sequential and limited for complex Seq2Seq tasks.

3.Encoder–Decoder (RNN/LSTM/GRU) → enabled variable-length input/output by separating encoding and decoding.

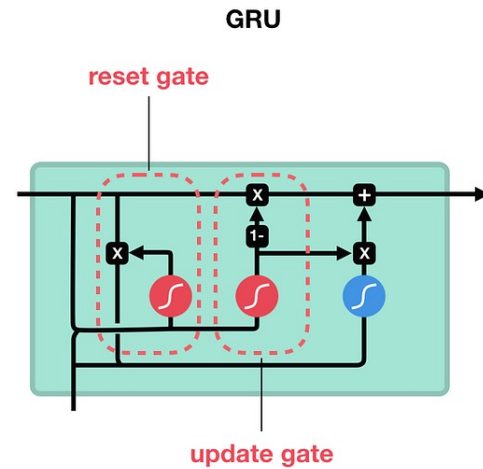
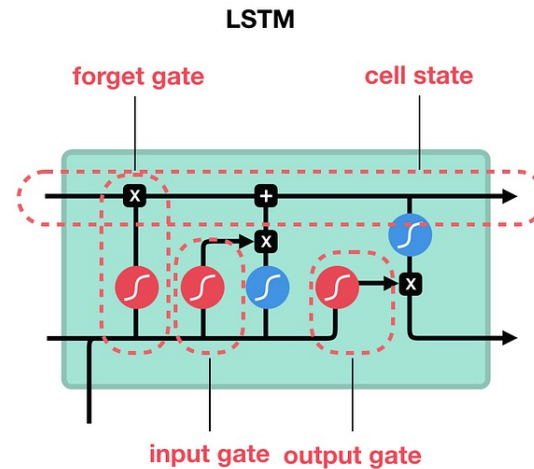
4.Encoder–Decoder + Attention → solved the single-vector bottleneck by letting the decoder attend to all encoder states.

5.Transformers → made attention the core computation, removing recurrence and enabling massive parallelization and scalability.

RNN unit

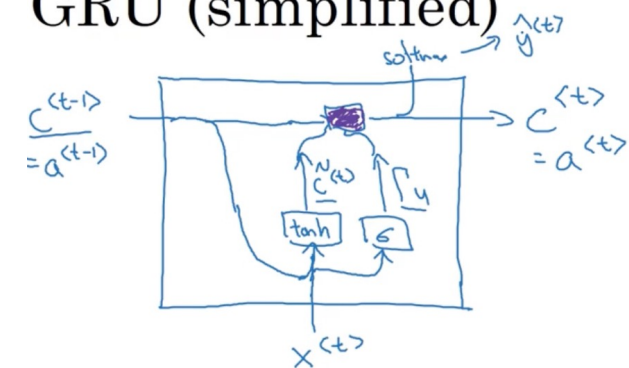


$$\underline{a^{<t>}} = g(\underbrace{W_a[a^{<t-1>}, x^{<t>}]}_{\text{tanh}} + \underline{b_a})$$



**vector
concatenation**

GRU (simplified)


$$\rightarrow C^{(t)} = Q^{(t)}$$

$$\rightarrow \hat{C}^{(t)} = \tanh(W_c [C^{(t-1)}, x^{(t)}])$$

$$\rightarrow \Gamma_y = \sigma(W_y [c^{(t-1)}, x^{(t)}])$$

$$\{ \underline{C}^{(t)} = \underbrace{u}_{=1} \times \underbrace{C^{(t)}}_{\text{poison}} + \underbrace{(1-u)}_1$$

Gate

LSTM in pictures

$$\tilde{c}^{<t>} = \tanh(W_c[a^{<t-1>}, x^{<t>}] + b_c)$$

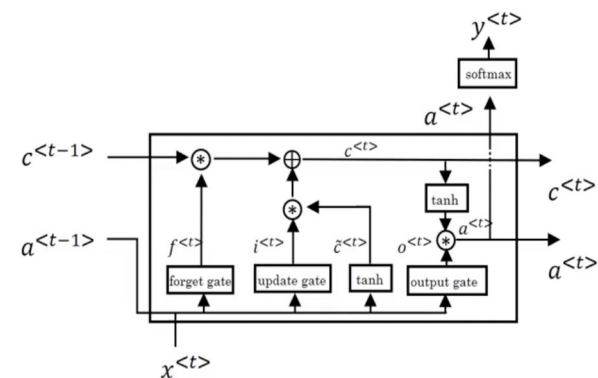
$$\Gamma_u = \sigma(W_u[a^{<t-1>}, x^{<t>}] + b_u)$$

$$\Gamma_f = \sigma(W_f[a^{<t-1>}, x^{<t>}] + b_f)$$

$$\Gamma_o = \sigma(W_o[a^{<t-1>}, x^{<t>}] + b_o)$$

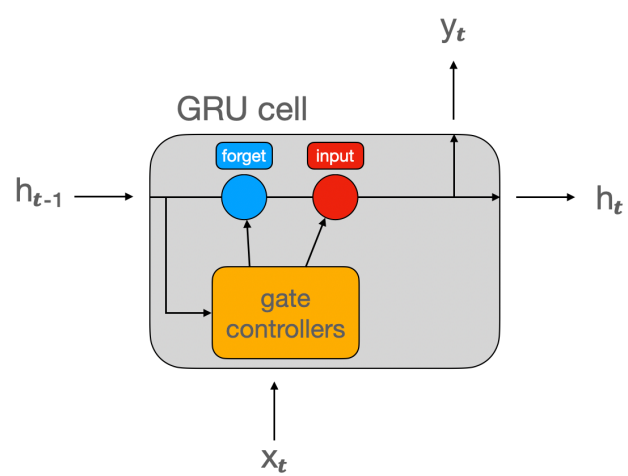
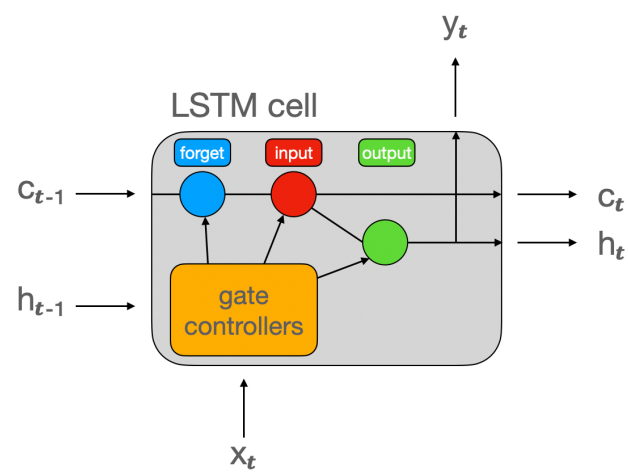
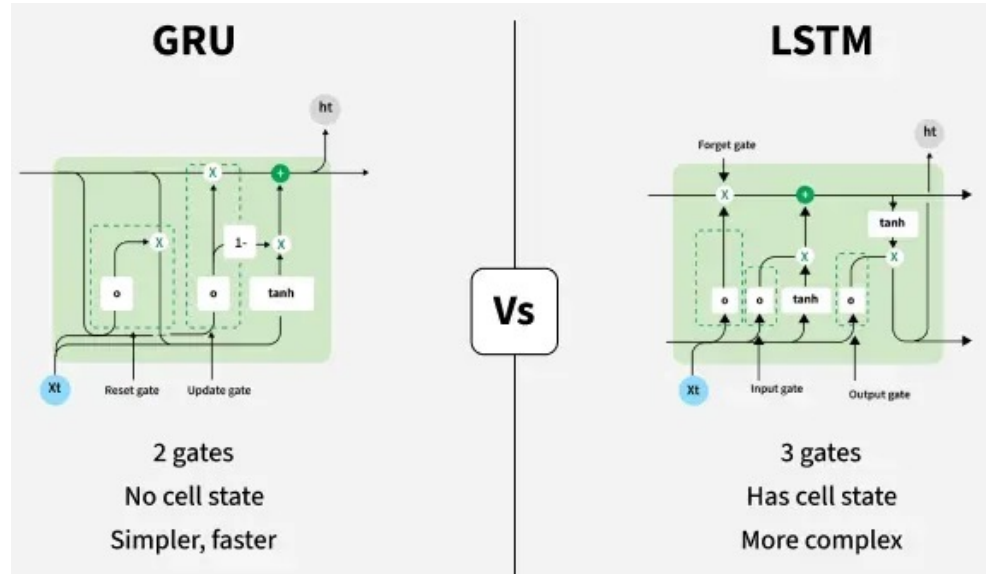
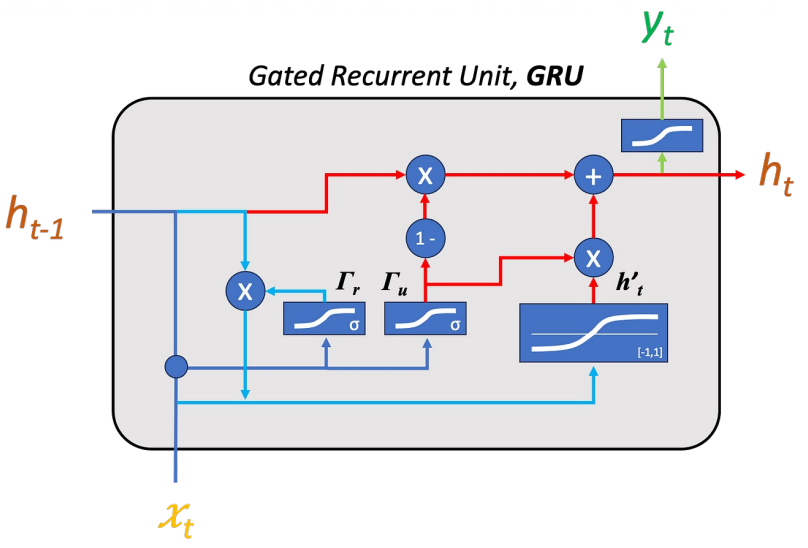
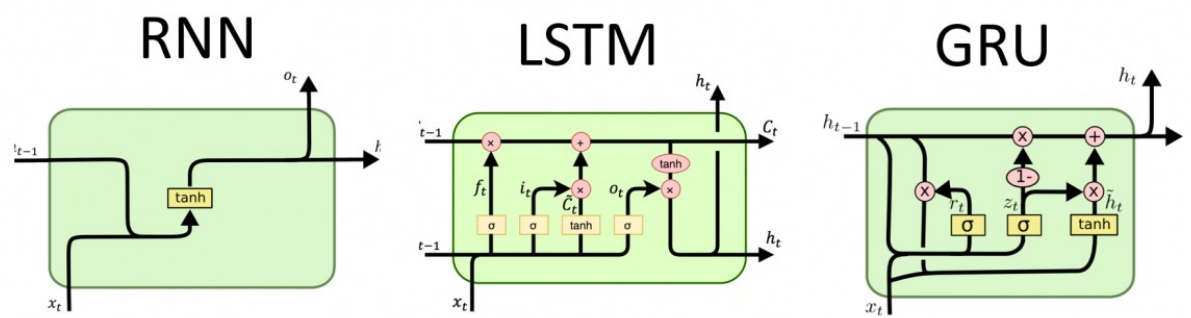
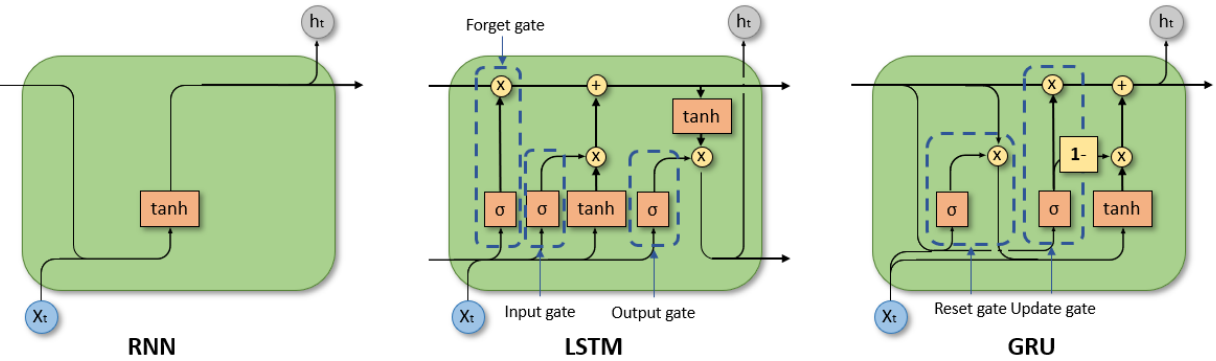
$$c^{<t>} = \Gamma_u * \tilde{c}^{<t>} + \Gamma_f * c^{<t-1>}$$

$$a^{<t>} = \Gamma_0 * \tanh c^{<t>}$$

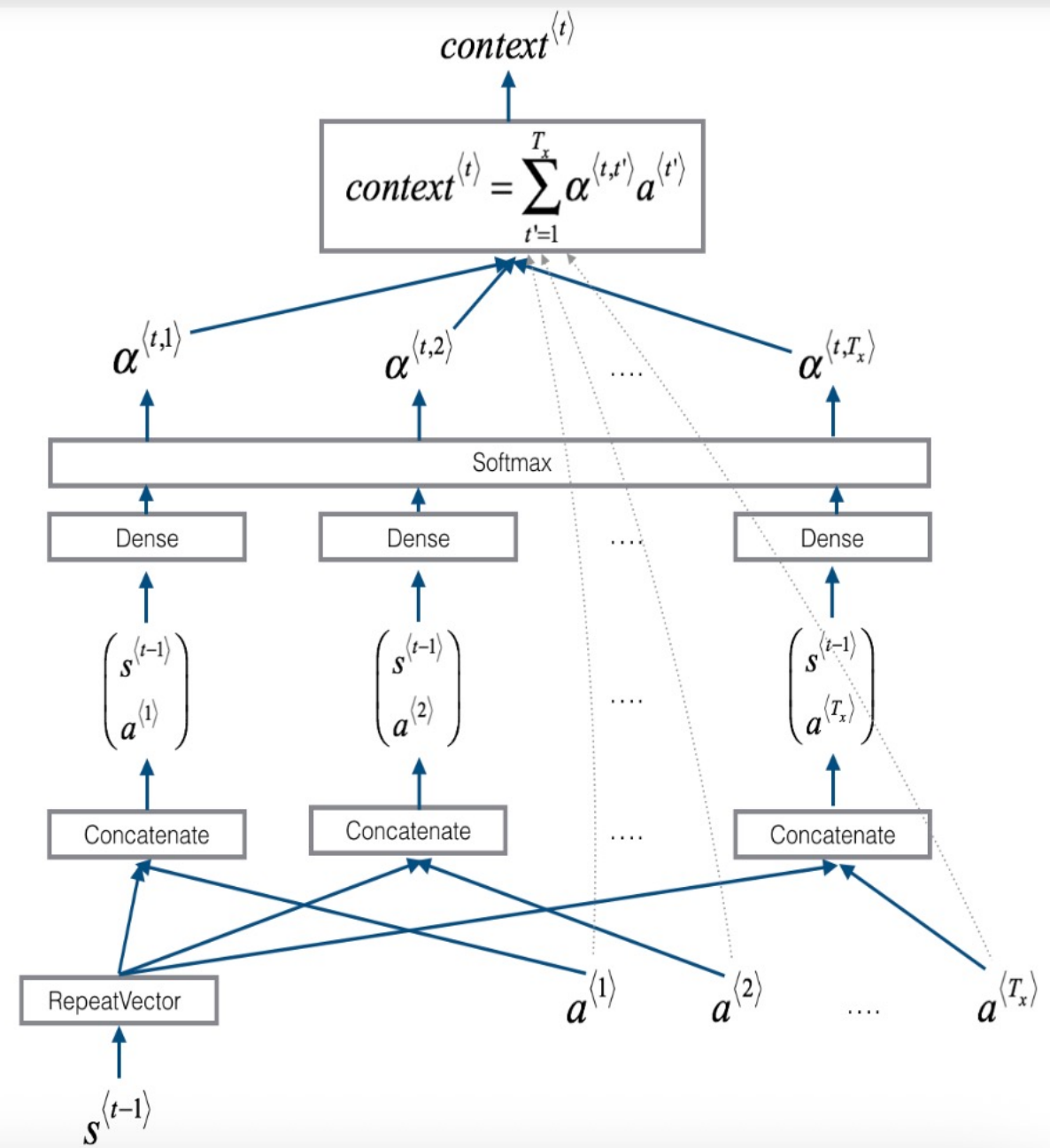
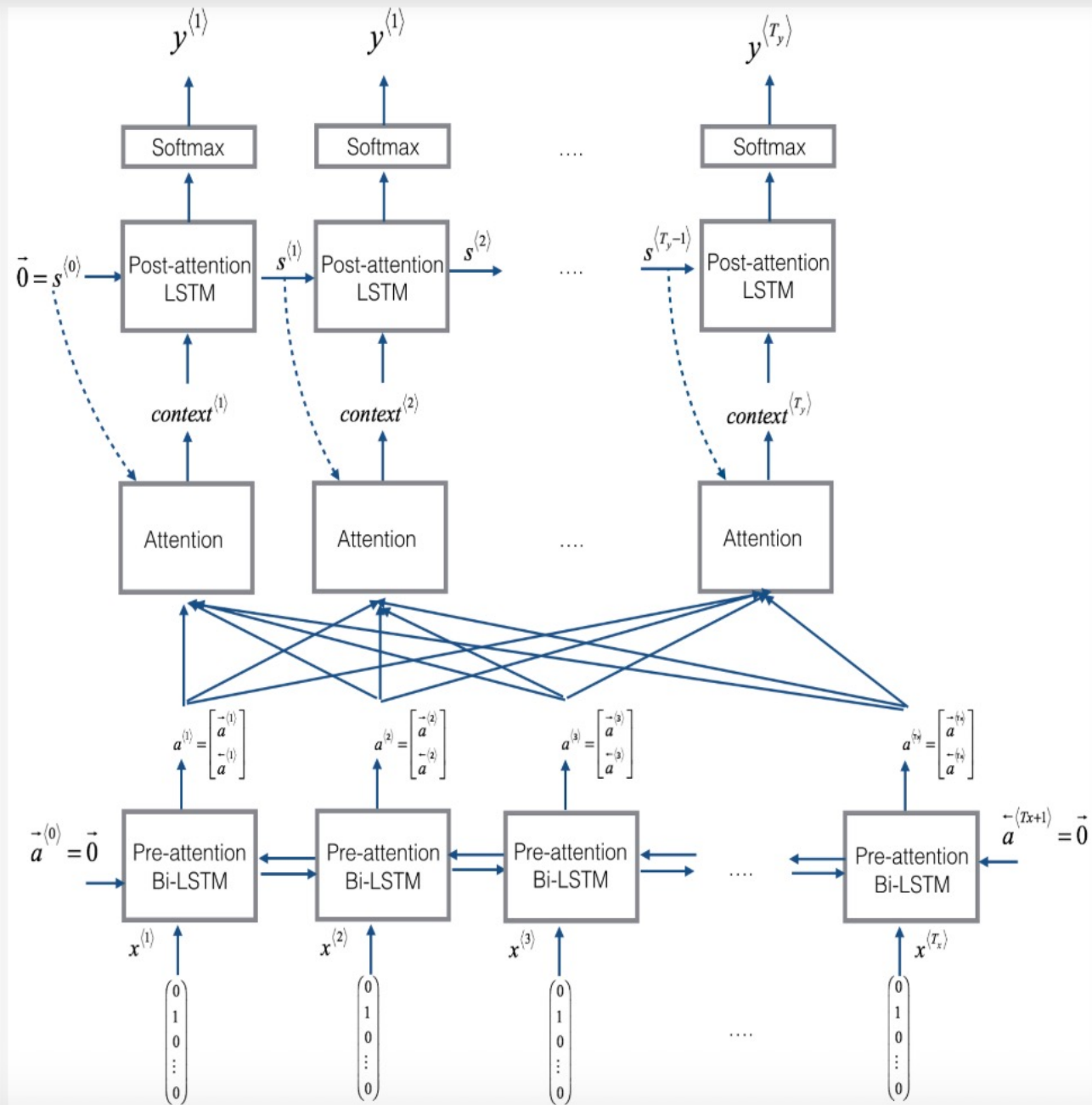


$\Gamma_4=1$ $\Gamma_2=0$ $\Gamma_3=0$ $\Gamma_1=0$ $\downarrow=1$
 $\Rightarrow$ $C^{(k)}=1$ $\downarrow=1$
 $\rightarrow$ The cat which already ate was full

Recurrent Networks (RNN, LSTM, GRU)



Attention in Sequential (RNN/LSTM/GRU) Encoder-Decoder Models



Attention in Transformer (Encoder-Decoder) Models

